



کامبیز در نیویورک

چیزی از مهاجرت خانواده موسوی به اسپانیا نگذشته بود که لازم شد کامبیز به همراه پدرش برای انجام کاری به نیویورک سفر کند. هتل محل اقامت آن‌ها در نزدیکی پارک مرکزی و در محله منهتن قرار داشت. پدرش صبح‌ها از هتل بیرون می‌زد و شب‌ها باز می‌گشت و کامبیز بیشتر اوقات تنها بود. او وقتش را یا در هتل می‌گذراند یا در کوچه پس کوچه‌های منهتن قدم می‌زد. او حین قدم زدن متوجه شد که در آنجا از ماشین‌های خودران زیاد استفاده می‌شود. از طرفی هنوز باورش نشده بود که ناگهان تا این حد پولدار شده‌اند؛ به همین دلیل تصمیم گرفت یک ماشین خودران بخرد و آن را به یک تاکسی خودران تبدیل کند تا هم در مخارج سفر به پدرش کمک کرده باشد و هم تمام وقت خود را وقف کار نکرده باشد.

پس از خرید خودرو متوجه شد که نمی‌تواند از مسیریاب پیش‌فرض خود خودرو برای تاکسی‌اش استفاده کند زیرا این مسیریاب برای اهداف تجاری ساخته نشده بود و نمی‌توانست از آن سود تجاری ببرد. پس خودش دست به کار شد تا یک مسیریاب اختصاصی برای تاکسی‌اش طراحی کند. اکنون همه چیز آماده و برنامه طراحی شده است. فقط طراحی الگوریتم مسیریابی باقی مانده است. از آنجا که سفر کامبیز رو به اتمام است و می‌خواهد کارها را موازی پیش ببرد، از شما می‌خواهیم الگوریتم‌های آگاهانه و ناآگاهانه زیر را برای او پیاده‌سازی کنید:

1. BFS (Breadth-First Search)
2. DLS (Depth-Limited Search)
3. BDS (Bidirectional Search)
4. A*

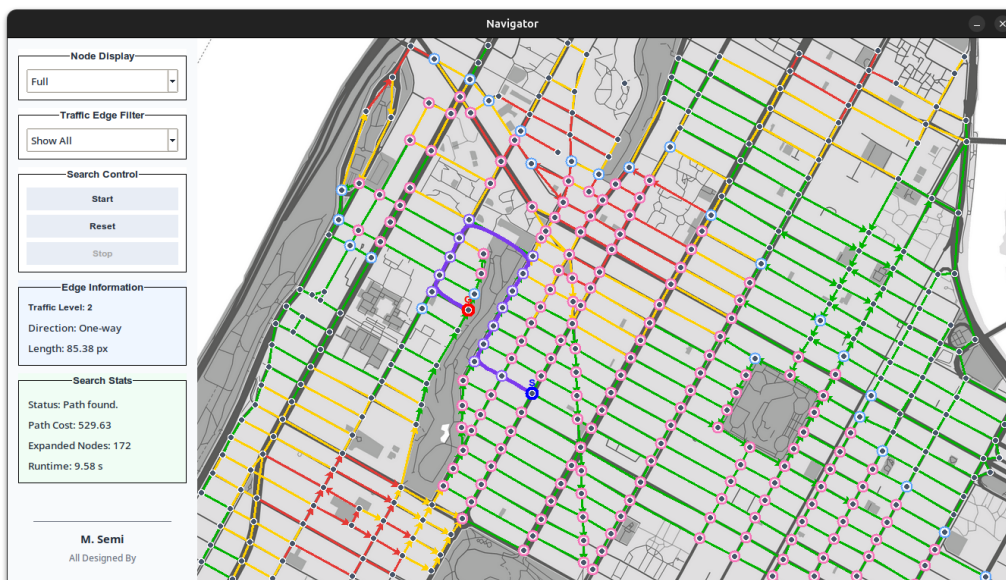
نکته: برای جست‌وجوی عمق محدود (DLS) می‌توانید یک حداکثر عمق معقول تخمین بزنید یا از الگوریتم IDS استفاده کنید.

در پایان مشخص شد که پدر کامبیز متأسفانه وارد تجارت مواد مخدر شده است و این سفر پدر و پسری به نیویورک تنها پوششی برای ملاقات او با رؤسای کارتل مواد مخدر بوده است!

محیط

در این پروژه محیط، که یک برنامه مسیریابی است، به طور کامل توسط کامبیز پیاده‌سازی شده و تنها بخش الگوریتم جست‌وجو باقی مانده است که شما باید با تعامل با محیط آن را پیاده‌سازی کنید. یک پوشه شامل فایل‌های پروژه در اختیار شما قرار می‌گیرد. فایل `environment.py` همان فایل محیط است که شما نباید هیچ تغییری در آن ایجاد کنید. برای اجرای آن به تعدادی کتابخانه نیاز خواهید داشت که نام آن‌ها در ابتدای فایل مشخص شده است و برای نصب‌شان می‌توانید از Mirrorهای موجود استفاده کنید.

فایل `main.py` فایلی است که شما باید الگوریتم‌های جست‌وجوی خود را در آن پیاده‌سازی کنید. در مورد نحوه پیاده‌سازی و تعامل با محیط در بخش بعد صحبت خواهیم کرد. با اجرای این فایل محیط برنامه قابل مشاهده خواهد بود، اما تا زمانی که الگوریتم جست‌وجوی شما کامل نشده و آن را به محیط ارسال نکرده باشید نمی‌توانید مسیریابی را آغاز کنید. از بخش Search Control در پنل سمت چپ می‌توانید اجرای برنامه و الگوریتم خود را مدیریت کنید.



شکل ۱: نمونه‌ای از محیط در حال اجرا

نقشه شامل خیابان‌های بخشی از محله منهتن نیویورک است. هر خیابان دارای یک میزان ترافیک است که شدت آن با اعداد ۱، ۲ و ۳ مشخص شده و به ترتیب با یال‌های سبز، زرد و قرمز نمایش داده می‌شود. ترافیک به صورت تصادفی تولید می‌شود اما می‌توانید با انتخاب یک مقدار مشخص برای Seed ترافیک‌ها را ثابت نگه دارید. دقت کنید که با این وجود توزیع ترافیک همیشه از واقعیت پیروی می‌کند.

برخی از خیابان‌ها یک‌طرفه هستند و عبور از آن‌ها تنها در یک جهت مجاز است. این خیابان‌ها و جهت مجازشان با یال‌های جهت‌دار در نقشه مشخص شده‌اند. با کلیک روی هر یال، می‌توانید از طریق پنل سمت چپ و بخش Edge Information اطلاعات مربوط به آن را مشاهده کنید. همچنین می‌توانید از قسمت Traffic Edge Filter فیلترهایی را روی نمایش یال‌ها اعمال کنید.

روی هر تقاطع یک گره خاکستری قرار دارد که با کلیک روی آن می‌تواند به عنوان مبدأ یا مقصد مسیر انتخاب شود. شما می‌توانید از قسمت Node Display نحوه نمایش گره‌ها را تغییر دهید تا دید بهتری از نقشه داشته باشید. همچنین برای راحتی کار، ابزار ذره‌بین برای زوم و امکان جابه‌جایی روی نقشه تعبیه شده است. با انتخاب یک گره به عنوان مبدأ و گره‌ای دیگر به عنوان مقصد، در صورتی که الگوریتم جست‌وجو آماده باشد می‌توانید مسیریابی را آغاز کنید. پس از پایان مسیریابی، در بخش Search Stats در پنل می‌توانید اطلاعات جامعی از نتیجه جست‌وجو کسب کنید. هزینه یک مسیر از جمع هزینه عبور از تک‌تک یال‌ها تا رسیدن به مقصد به دست می‌آید. هزینه عبور از یک یال نیز برابر با حاصل ضرب طول یال در شدت ترافیک آن است.

گسترش فضای جست‌وجو به شکل یک انیمیشن نمایش داده می‌شود. دور گره‌های مشاهده شده (Explored) هاله صورتی و دور گره‌های مرزی (Frontier) هاله آبی رنگ ایجاد می‌شود. در نهایت، مسیر پیدا شده با رنگ بنفش مشخص شده و گره‌هایی که در ساخت آن نقش داشته‌اند هاله‌ای به همین رنگ به خود می‌گیرند. سرعت نمایش انیمیشن نیز با استفاده از پارامتر delay در سازنده کلاس Map قابل تنظیم است.

نحوه پیاده‌سازی

شما باید الگوریتم‌های خواسته شده را در فایل main.py پیاده‌سازی کنید. قالب کلی پیاده‌سازی از پیش در این فایل آورده شده است. برای تسهیل کار، متدهای مفیدی برای تعامل با محیط پیاده‌سازی شده‌اند. این متدها روی شیء کلاس Map عمل می‌کنند. در این پروژه هر وضعیت (state) به صورت یک تابل (tuple) دوتایی از مختصات گره در نظر گرفته شده است. برای تعامل با محیط صرفاً مجاز به استفاده از متدهای زیر هستید:

۱. `tuple -> env.get_start_state()`: گره شروع مسیر را برمی‌گرداند.

۲. `list -> env.get_successors(state)`: لیستی از گره‌های مجاور را که می‌توان از گره فعلی به آن‌ها رفت به همراه هزینه رفتن به هر یک برمی‌گرداند.

۳. `bool -> env.is_goal_state(state)`: مشخص می‌کند که آیا گره فعلی مقصد است یا خیر.

۴. `tuple -> env.get_goal_state()`: گره مقصد را برمی‌گرداند. دقت کنید که تنها در الگوریتم‌های BDS و A^* مجاز به استفاده از این متد هستید.

۵. `list -> env.get_predecessors(state)`: لیستی از گره‌های مجاور را که می‌توان از آن‌ها به گره فعلی آمد به همراه هزینه آمدن از هر یک برمی‌گرداند. تنها در الگوریتم BDS مجاز به استفاده از این متد هستید.

برای اجرای درست انیمیشن‌ها لازم است در تابع خود، وضعیت‌های جدید را به محیط اعلام کنید. لیست وضعیت‌ها و کاربرد آن‌ها در زیر آمده است:

۱. `yield ("expand", state)`: هرگاه قصد دارید یک گره را گسترش دهید باید از این ژنراتور استفاده کنید. برای مثال در جایی که وضعیت فعلی (`current_state`) را مقداردهی می‌کنید احتمالاً قصد دارید در ادامه آن را گسترش دهید؛ پس منطقی است که پس از مقداردهی با این دستور، وضعیت یا گره فعلی را به عنوان گره گسترش یافته به محیط اعلام کنید.

۲. `yield ("goal", path)`: وقتی الگوریتم به مقصد رسید احتمالاً یک لیست از گره‌هایی را که به ترتیب مسیر مبدأ تا مقصد را تشکیل می‌دهند به عنوان جواب خروجی خواهید داد. پیش از خروجی دادن (`Return`) این مسیر را با این ژنراتور به محیط اعلام کنید.

۳. `yield ("frontier", successor)`: احتمالاً هنگام گسترش دادن یک گره کل یا برخی از گره‌های جانشین (`successor`) آن را به صف مرز (`Frontier`) اضافه خواهید کرد. پس از افزودن یک `successor` به `frontier` آن را به محیط نیز اعلام کنید.

۴. `yield ("reset_visuals", None)`: ممکن است بخواهید حین اجرای الگوریتم، عناصر گرافیکی تولید شده توسط الگوریتم روی نقشه را پاک کنید. مثلاً در پیاده‌سازی DLS با استفاده از IDS چندین DFS با عمق‌های متفاوت اجرا خواهیم کرد. بهتر است پیش از اجرای DFS بعدی، عناصر تولید شده توسط DFS فعلی را از روی نقشه پاک کنیم.

۵. `yield ("fail", None)`: در نهایت ممکن است کل فضای جست‌وجو را پیمایش کنیم اما پاسخی پیدا نشود. در این حالت اجرای الگوریتم بدون پاسخ به پایان می‌رسد. این حالت شکست را با این ژنراتور به محیط اعلام کنید.

نحوه ارزیابی

۱. در الگوریتم BFS باید تعداد گره‌های گسترش یافته و تعداد گره‌های سازنده مسیر کمینه باشند.

۲. در الگوریتم DLS باید تعداد گره‌های گسترش یافته منطقی و تعداد گره‌های سازنده مسیر کمینه باشد.

۳. در الگوریتم BDS باید تعداد گره‌های گسترش یافته و زمان اجرا به مراتب بهتر از BFS بوده و تعداد گره‌های سازنده مسیر کمینه باشد.

۴. در الگوریتم A^* ، باید تعداد گره‌های گسترش یافته تا حد معقولی اندک باشد و الگوریتم از سرعت خوبی برخوردار باشد. همچنین هزینه مسیر یافت شده باید کمینه باشد.

بخش امتیازی: در الگوریتم A^* ، علاوه بر موارد خواسته شده اگر تابع اکتشافی (`Heuristic`) خلاقانه‌ای نیز پیاده‌سازی کنید که با اندکی صرف نظر از بهینه بودن مسیر یافت شده با گسترش گره‌هایی کمتر از ۳ برابر گره‌های سازنده مسیر به پاسخ برسد نمره امتیازی دریافت خواهید کرد. به عبارت دیگر، در تمام حالت‌ها الگوریتم شما باید تعداد گره‌هایی را گسترش دهد که از سه برابر تعداد گره‌های سازنده مسیر کمتر باشد و در عین حال هزینه مسیر یافت شده معقول و تنها اندکی از مسیر بهینه بیشتر باشد. این بخش در صورت پیاده‌سازی درست و رعایت نکات گفته شده به تنهایی به اندازه یک چهارم نمره بخش اجباری نمره اضافه در پی خواهد داشت.

توجه کنید که درستی پیاده‌سازی و صحت عملکرد توابع شما به صورت جداگانه روی حالت‌های مختلف بررسی خواهد شد. توابع شما باید بتوانند در نقشه‌های مختلف به کار گرفته شوند؛ نقشه‌هایی که می‌توانند از نظر توزیع ترافیک و یک‌طرفه بودن خیابان‌ها متفاوت باشند. انتظار می‌رود تعداد گره‌های بسط داده شده در حد قابل قبولی باشد. شما می‌توانید برای سنجش عملکرد پیاده‌سازی‌های خود حالت‌های مختلفی در محیط بسازید و روی آن‌ها آزمایش کنید.

نکات پایانی

۱. تمام توابع خود را در فایل `main.py` پیاده‌سازی کنید و پیش از ارسال نام آن را به شماره دانشجویی خود تغییر دهید (برای مثال: `4003613001.py`). در غیر این صورت ممکن است بخشی از نمره خود را از دست بدهید. در نهایت فایل را در کوئرا و در تمرینی با نام «پیاده‌سازی» بارگذاری کنید.

۲. پس از پایان پیاده‌سازی گزارشی تهیه کرده و در آن کدهای خود را توضیح دهید و نتایج را به نمایش بگذارید. گزارش را در قالب یک فایل pdf به همراه کدها تحویل دهید. برای قالب گزارش می‌توانید از قالب دانشکده استفاده کنید. پیش از ارسال نام فایل گزارش را نیز به شماره دانشجویی خود تغییر دهید (برای مثال: `4003613001.pdf`). در غیر این صورت ممکن است بخشی از نمره خود را از دست بدهید. فایل نهایی را در کوئرا و در تمرینی با نام «گزارش» بارگذاری کنید.

۳. برای ارائه باید روی تمام بخش‌های پیاده‌سازی خود مسلط باشید، دلیل و منطق استفاده از روش‌هایتان را بدانید و این موارد را در گزارش خود نیز ذکر کرده باشید.

با استفاده از این ابزار می‌توانید عملکرد هر یک از الگوریتم‌های جست‌وجو را به صورت دقیق بررسی کنید و با ماهیت هر یک به خوبی آشنا شوید.

با آرزوی سلامت و بهروزی، موفق باشید.